

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

Курсовое проектирование по управлению ресурсами суперэвм

ОТЧЁТ ПО КУРСОВОЙ РАБОТЕ

«Возведение матрицы в k -ю степень»

Студент,
группы 5130201/30102

_____ Михайлова А. А.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Аппаратно программная платформа Nvidia CUDA	5
2.1 Архитектура Nvidia CUDA	5
2.1.1 Архитектура Blackwell (март 2024)	6
2.1.2 Архитектура Horner (март 2022)	7
2.1.3 Архитектура Ады Лавлейс (сентябрь 2022)	8
2.1.4 Архитектура Ampere (2020)	10
2.1.5 Архитектура Pascal (2016)	11
2.1.6 Архитектура Maxwell (2014)	13
2.1.7 Архитектура Tesla (2006)	14
2.2 Вычислительные возможности Nvidia CUDA	16
2.3 Поточковая модель	18
2.4 Устройство памяти	19
2.5 Модели памяти	19
2.6 Модель вычислений на GPU	21
2.7 Компиляция программы	21
2.8 Планировщик задач	22
3 Суперкомпьютерный центр «Политехнический»	23
3.1 Состав	23
3.2 Характеристики	23
3.3 Технология подключения	24
4 Постановка решаемой практической задачи	27
5 Алгоритм решения задачи	28
5.1 Метод распараллеливания алгоритма	28
5.1.1 Глобальная память	28
5.1.2 Разделяемая память	29
6 Описание эксперимента	31
7 Анализ результатов	32
Заключение	41
Список использованной литературы	42

Введение

Сейчас многие научные и инженерные задачи требуют больших мощностей машин, на которых они выполняются, так как в ходе их выполнения необходимо обрабатывать огромное число математических операций за приемлемое время. Одним из вариантов решения сложных вычислительных задач является использование параллельного программирования.

Суперкомпьютеры представляют собой многопроцессорные вычислительные комплексы, способные выполнять триллионы операций в секунду. Их использование позволяет решать задачи, недоступные для традиционных последовательных вычислений. Это особенно актуально для таких областей, как численное моделирование, обработка изображений, машинное обучение. В связи с этим в последние годы стало очень распространено вычисление с помощью графических ускорителей — устройств с массивно-параллельной архитектурой. Производить вычисления общего назначения можно на видеокартах архитектуры Nvidia CUDA.

Одним из примеров массивных вычислительных систем является суперкомпьютерный центр «Политехнический». Часть узлов этого суперкомпьютера оборудована графическими ускорителями Nvidia Tesla K40X.

1 Постановка задачи

Цель курсовой работы: исследование технологий параллельного программирования на основе архитектуры NVIDIA CUDA для решения задачи возведения матрицы в k -тую степень с использованием вычислительных ресурсов СКЦ «Политехнический».

В рамках поставленной цели необходимо выполнить следующие задачи:

1. Изучить аппаратно-программную платформу NVIDIA CUDA: её архитектуру, вычислительные возможности, устройство памяти, модель вычислений.
2. Изучить состав и характеристики вычислительных средств СКЦ «Политехнический», изучить и применить технологию подключения к СКЦ.
3. Реализовать с использованием технологии CUDA C алгоритм возведения матрицы в k -тую степень.
4. Провести серию вычислительных экспериментов на узле «Торнадо» СКЦ «Политехнический» с изменением следующих параметров:
 - количество потоков в блоке,
 - количество блоков,
 - размер исходной матрицы,
 - используемые модели матрицы.
5. Провести анализ полученных результатов и сравнить время выполнения реализации при различных параметрах.

2 Аппаратно программная платформа Nvidia CUDA

NVIDIA CUDA (Compute Unified Device Architecture) представляет собой совокупность программных и аппаратных средств, разработанную компанией NVIDIA и предназначенную для организации параллельных вычислений на графических процессорах (GPU), а также для ускорения обработки информации.

2.1 Архитектура Nvidia CUDA

Архитектура графического процессора (GPU) включает:

- **GPU-процессор** — представляет собой массив потоковых процессоров (Streaming Processor Array).
- **Потоковый процессор** — состоит из кластеров текстурных процессоров (Texture Processor Clusters).
- **Текстурные процессоры** — в свою очередь, состоят из набора мультипроцессоров (Streaming Multiprocessors, SM).
- **Мультипроцессоры** — содержат несколько потоковых процессоров (Streaming Processors), которые также называют ядрами CUDA (CUDA cores).

Управление выполнением команд осуществляется с помощью GigaThread Engine, который распределяет блоки потоков между мультипроцессорами. Общая структура GPU не претерпевает кардинальных изменений при переходе от одной микроархитектуры к другой — в основном варьируются объём и скорость L2-кэша.

В архитектуре NVIDIA CUDA используется модель исполнения SIMT (Single Instruction Multiple Thread — одиночная инструкция, множество потоков). Эта модель сочетает черты MIMD (Multiple Instruction Multiple Data) и SIMD (Single Instruction Multiple Data). Вычислительная задача разбивается на множество блоков. Планировщик назначает блоки на мультипроцессоры (SM), причём каждый блок выполняется только на одном SM, однако на одном SM может одновременно выполняться несколько блоков.

Каждый блок состоит из потоков (threads). При выполнении потоки группируются в варпы (warps) — коллекции по 32 потока. Все потоки внутри одного варпа одновременно исполняют одну и ту же команду.

2.1.1 Архитектура Blackwell (март 2024)

Архитектура Blackwell представляет собой значительный шаг вперед в развитии графических технологий от NVIDIA, охватывая три ключевых аспекта применения: гейминг, студийные задачи и искусственный интеллект. Рассмотрим подробно каждый аспект, подчеркивая уникальные технологии и преимущества архитектуры.

Видеокарты серии Blackwell уже доступны в продаже. Надёжным партнёром по их поставке выступает интернет-магазин Telemart, предлагающий широкий ассортимент моделей NVIDIA GeForce RTX 50-й серии, включая как игровые, так и профессиональные решения.

Архитектура Blackwell делает особый акцент на реалистичную визуализацию человеческой кожи в играх. Технология RTX Skin использует усовершенствованные алгоритмы подповерхностного рассеивания света (SSS), что позволяет добиться правдоподобной передачи полутонов, текстур и цветовых переходов. В отличие от предыдущих реализаций, система теперь учитывает не только основной свет, но и контекстные изменения освещения — в том числе влияние окружающей среды и отражённого света от других поверхностей.

Особое внимание уделено адаптивным алгоритмам, которые в реальном времени настраивают интенсивность и глубину рассеивания света под кожей. Это позволяет сохранить корректное отображение даже при резкой смене условий — например, при переходе из солнечного пространства в тень или при использовании источников холодного направленного света. Эффект особенно заметен в сценах с близкими планами, где важны микродетали текстуры кожи — поры, сосуды, тональные переходы.

Первое практическое применение RTX Skin можно увидеть в Half-Life 2 RTX Remaster, где технология используется для обновлённого отображения лиц главных персонажей. В катсценах и диалогах заметно, как кожа реагирует на освещение от фонарей, солнечного света или светящихся экранов. Такой уровень фотореализма делает старую игру визуально современнее и подчёркивает потенциал RTX Skin в кинематографичных проектах нового поколения.

DLSS 4 с Frame Generation и Multi-Frame Generation

DLSS 4 представляет собой масштабное обновление технологии Deep Learning Super Sampling от NVIDIA, анонсированное одновременно с архитектурой Blackwell и видеокартами серии GeForce RTX 50. Основными нововведениями стали использование модели трансформера и интеграция Multi Frame Generation, благодаря которым достигается значительный прирост производительности и резкое улучшение визуального качества — особенно в динамичных сценах и при высоких разрешениях.

Новая система генерации кадров теперь способна учитывать не только текущий и предыдущий кадры, но и пространственно-временные зависимости между несколькими отрезками рендеринга, что минимизирует артефакты и

повышает чёткость изображения. DLSS 4 также задействует более сложную нейросетевую архитектуру, позволяющую адаптироваться к конкретной игре и сценарию, обеспечивая более стабильный результат.

2.1.2 Архитектура Nopper (март 2022)

Архитектура Nopper представляет собой высокопараллельную вычислительную платформу, спроектированную на базе графического процессора GH100. Структура архитектуры опирается на сложную иерархическую организацию вычислительных блоков и глубоко интегрированную подсистему памяти, ориентированную на максимизацию локальности данных.

Вычислительное ядро и потоковые мультипроцессоры (SM)

Базовым строительным блоком архитектуры выступает потоковый мультипроцессор (Streaming Multiprocessor, SM). Каждый SM содержит независимые конвейеры для целочисленных вычислений (INT32) и вычислений с плавающей запятой, а также специализированные блоки матричной математики.

- **Тензорные ядра (Tensor Cores):** Вычислительные блоки, аппаратно реализующие операции умножения и сложения матриц (Matrix Multiply-Accumulate). Они аппаратно поддерживают вычисления смешанной точности, включая форматы FP8, FP16, bfloat16, TF32 и FP64.
- **Transformer Engine:** Аппаратный модуль, функционирующий совместно с тензорными ядрами на уровне SM. Он выполняет эвристический анализ распределения значений в тензорах на каждом слое нейронной сети и осуществляет динамическое (на лету) переключение точности вычислений между форматами FP8 и FP16, оптимизируя информационную энтропию и пропускную способность.
- **Инструкции DPX:** Интегрированный в арифметико-логические устройства (ALU) набор инструкций, реализующий аппаратное ускорение рекурсивных алгоритмов динамического программирования на уровне отдельных тактов процессора.

Иерархия памяти и асинхронная передача данных

Для предотвращения простоя вычислительных блоков (data starvation), архитектура Nopper включает специализированные механизмы управления памятью:

- **Тензорный ускоритель памяти (Tensor Memory Accelerator, TMA):** Выделенный аппаратный блок управления прямым доступом к памяти. TMA берет на себя задачи асинхронного копирования многомерных блоков данных между глобальной памятью (HBM3) и разделяемой памятью

(Shared Memory). Он вычисляет адреса и управляет транзакциями аппаратно, полностью высвобождая регистры и ALU потокового мультимикропроцессора.

- **Распределенная разделяемая память (Distributed Shared Memory, DSMEM):** Архитектурная инновация, позволяющая объединять пулы локальной разделяемой памяти нескольких SM. Микропроцессоры могут напрямую читать и писать данные в L1/Shared память соседних SM по выделенному высокоскоростному интерконнекту, минуя обращение к кэшу второго уровня (L2) или глобальной памяти.

Иерархия планирования: Thread Block Clusters

В архитектуре Hopper традиционная иерархия планирования потоков (Grid → Block → Thread) расширена новым промежуточным уровнем абстракции — кластерами блоков потоков (*Thread Block Clusters*). Эта парадигма позволяет группировать несколько блоков потоков на этапе компиляции и аппаратно гарантировать их совместное (кооперативное) планирование на физически соседних SM. Потоки внутри одного кластера получают возможность аппаратной барьерной синхронизации и прямого использования технологии DSMEM, что позволяет обрабатывать крупные фрагменты графов вычислений локально, не выгружая промежуточные результаты в общую память устройства.

2.1.3 Архитектура Ады Лавлейс (сентябрь 2022)

Архитектура Ada Lovelace представляет собой вычислительную платформу, аппаратно оптимизированную для ресурсоемкого графического рендеринга, симуляции физики света и нейросетевого инференса. Структурная парадигма архитектуры строится на глубокой специализации вычислительных блоков в рамках потоковых микропроцессоров и оптимизации путей передачи данных.

Вычислительное ядро и специализированные сопроцессоры

Фундаментальным вычислительным узлом является потоковый микропроцессор (SM), объединяющий блоки общего и узкоспециализированного назначения:

- **Ядра CUDA (Универсальные ALU):** Блоки, реализующие архитектуру параллельных вычислений и обеспечивающие одновременное (конкурентное) выполнение инструкций с плавающей запятой (FP32) и целочисленных операций (INT32).
- **Тензорные ядра (Tensor Cores):** Аппаратные матричные сопроцессоры, адаптированные для вывода нейросетей и обработки алгоритмов искусственного интеллекта. Они поддерживают расширенный спектр фор-

матов данных, включая тензорный формат FP8, что позволяет значительно повысить пропускную способность при сохранении необходимой точности в задачах глубокого обучения.

- **Ядра трассировки лучей (RT Cores):** Выделенные аппаратные блоки для расчетов пересечения лучей с ограничивающими объемами (BVH) и треугольниками. Включают интегрированные подмодули: *Opacity Micromap Engine* для аппаратного исключения расчетов прозрачной геометрии (например, листвы) и *Displaced Micro-Mesh Engine* для генерации высокодетализированной геометрии на аппаратном уровне без перегрузки основных конвейеров и подсистемы памяти.

Аппаратная перекомпоновка и медиа-движки

Для борьбы с нерегулярными вычислительными нагрузками в архитектуру внедрены специализированные аппаратные планировщики:

- **Shader Execution Reordering (SER):** Аппаратный механизм динамического переупорядочивания потоков (threads) на лету. В задачах с высокой дивергенцией (когда лучи отражаются в разных направлениях и требуют разных шейдеров), SER группирует потоки с аналогичными инструкциями или схожим доступом к памяти. Это превращает нерегулярную нагрузку в последовательную, максимизируя утилизацию вычислительных блоков мультипроцессора.
- **Ускоритель оптического потока (Optical Flow Accelerator, OFA):** Выделенный аппаратный блок машинного зрения, который анализирует последовательные кадры и вычисляет векторы движения пикселей. Опираясь на эти данные, тензорные ядра могут генерировать полностью новые кадры методами ИИ, минуя традиционный конвейер рендеринга.
- **Двойные аппаратные кодировщики (Dual NVENC):** Интегрированные медиа-процессоры с аппаратной поддержкой кодирования и декодирования формата AV1, способные разделять кадр на части для параллельной обработки.

Подсистема памяти и локализация данных

Архитектура памяти спроектирована с упором на максимальное удержание данных внутри чипа:

- **Масштабированный кэш второго уровня (L2 Cache):** В архитектуре реализован радикально увеличенный объем унифицированной кэш-памяти L2. Это структурное решение гарантирует, что массивы данных BVH, необходимые для трассировки лучей, и весовые коэффициенты нейросетей остаются в высокоскоростной памяти графического процессора.

Это минимизирует количество транзакций по шине к подсистеме видеопамяти (VRAM), снижая энергопотребление и задержки (latency) при доступе к данным.

2.1.4 Архитектура Ampere (2020)

Архитектура Ampere представляет собой унифицированную платформу, архитектурно сбалансированную для удовлетворения требований как традиционных графических рабочих нагрузок, так и интенсивных вычислений в области искусственного интеллекта (ИИ) и высокопроизводительных вычислений (HPC). Основной фокус архитектуры — повышение энергоэффективности, параллелизма и введение новых уровней точности вычислений.

Вычислительное ядро и потоковые мультипроцессоры (SM)

Потоковый мультипроцессор (SM) в архитектуре Ampere подвергся значительной переработке для обеспечения большей гибкости конвейеров:

- **Разделенные пути данных (Datapaths):** Архитектура внедряет параллельные конвейеры для выполнения инструкций. Один путь выделен исключительно для операций с плавающей запятой (FP32), в то время как второй может динамически переключаться между целочисленными операциями (INT32) и операциями FP32 в пределах одного такта. Это удваивает теоретическую пиковую производительность FP32 по сравнению с архитектурами предыдущих поколений.
- **Тензорные ядра (Tensor Cores):** Аппаратные блоки матричного умножения оснащены поддержкой новых форматов данных: *TF32 (Tensor Float 32)* и *Bfloat16*. Формат TF32 позволяет обрабатывать данные FP32 с диапазоном FP32, но с точностью FP16, что обеспечивает значительное ускорение обучения нейросетей без модификации программного кода.
- **Механизм структурной разреженности (Structural Sparsity):** Тензорные ядра получили аппаратную поддержку разреженных матриц с соотношением 2:4 (на каждые четыре значения в векторе аппаратура ожидает два нулевых). Оборудование автоматически сжимает данные и пропускает вычисления с нулями, удваивая математическую пропускную способность и снижая требования к пропускной способности памяти при инференсе нейронных сетей.
- **Ядра трассировки лучей (RT Cores):** Аппаратные блоки, выполняющие параллельные вычисления пересечений лучей с треугольниками и ограничивающими объемами (BVH). В архитектуре реализовано одновременное выполнение вычислений трассировки лучей, операций рендеринга (затенения) и тензорных вычислений.

Иерархия памяти и управление данными

Подсистема памяти Ampere спроектирована для обеспечения непрерывного потока данных к высокопроизводительным SM:

- **Асинхронное копирование (Asynchronous Copy):** На уровне SM реализованы новые инструкции для асинхронного копирования данных непосредственно из глобальной памяти в разделяемую память (Shared Memory), минуя регистровый файл. Это высвобождает регистры и снимает нагрузку с ALU во время транзакций памяти.
- **L2 Cache Residence Controls:** Архитектура предоставляет программируемые элементы управления на уровне кэша L2. Это позволяет явно назначать фрагментам данных (например, часто используемым матрицам или структурам BVH) приоритет для постоянного хранения (residency) в кэше L2, минимизируя дорогостоящие обращения к глобальной памяти.

Мультиарендность и виртуализация

Для корпоративных сред и центров обработки данных в архитектуру внедрен аппаратный механизм разделения ресурсов:

- **Multi-Instance GPU (MIG):** Аппаратная технология, позволяющая изолировать и разбивать один физический графический процессор на несколько (до семи) независимых инстансов. Каждый инстанс получает гарантированные и полностью изолированные на аппаратном уровне ресурсы: сегменты кэша L2, пропускную способность памяти, потоковые мультипроцессоры и блоки кодирования. Это обеспечивает строгую изоляцию отказов (fault isolation) и предсказуемую производительность при параллельном выполнении разнородных задач.

2.1.5 Архитектура Pascal (2016)

Архитектура Pascal представляет собой фундаментальный этап в эволюции графических процессоров, ознаменовавший переход к аппаратному ускорению высокопроизводительных вычислений (HPC) и алгоритмов глубокого обучения. В ее основе лежат радикальные изменения в организации памяти, коммуникационных интерфейсах и вычислительной точности.

Вычислительное ядро и потоковые мультипроцессоры (SM)

Потоковый мультипроцессор в архитектуре Pascal (в частности, в старшем ядре GP100) был структурно реорганизован для повышения эффективности планирования инструкций и максимизации утилизации регистров:

- **Оптимизированная структура SM:** Мультипроцессор разделен на два независимых вычислительных блока (вместо четырех в архитектуре Maxwell), каждый из которых имеет свой собственный буфер инструкций,

планировщик (warp scheduler) и диспетчер. Это упрощает логику диспетчеризации и увеличивает долю площади кристалла, отводимой непосредственно под арифметико-логические устройства (ALU) и регистровый файл.

- **Аппаратная поддержка смешанной точности (Mixed-Precision Computing):** Архитектура впервые внедрила нативную аппаратную поддержку вычислений с половинной точностью (FP16). Инструкции FP16 могут упаковываться попарно (векторизация) и выполняться на ALU одинарной точности, что позволяет удвоить пропускную способность по сравнению с FP32. Это стало критически важным механизмом для ускорения обучения нейронных сетей.

Инновации в подсистеме памяти и адресации

Для преодоления ограничений пропускной способности (решения проблемы «стены памяти») архитектура Pascal использует передовые технологии физической компоновки и виртуализации данных:

- **Память HBM2 (High Bandwidth Memory):** Использование технологии 2.5D компоновки, при которой многослойные стеки памяти размещаются на одной кремниевой подложке (interposer) с графическим процессором. Это решение позволило многократно увеличить пропускную способность памяти (до 720 ГБ/с в первых реализациях) и существенно снизить задержки по сравнению с дискретной памятью GDDR5.
- **Унифицированная память и аппаратный страничный обмен (Page Migration Engine):** Внедрение выделенного аппаратного контроллера, поддерживающего обработку ошибок отсутствия страницы памяти (page faults) со стороны GPU. Это обеспечило создание истинно единого виртуального адресного пространства между CPU и GPU. При обращении графического процессора к данным, находящимся в оперативной памяти хоста, контроллер автоматически, страница за страницей, мигрирует необходимые данные в локальную память VRAM по требованию (on-demand), абстрагируя программиста от ручного управления пересылками.

Механизмы коммуникации и планирования задач

Архитектура пересмотрела подходы к масштабированию многопроцессорных систем и разделению вычислительных ресурсов:

- **Высокоскоростной интерконнект NVLink:** Интеграция первого поколения проприетарной шины для когерентного и прямого взаимодействия между графическими процессорами (GPU-to-GPU), минуя корневой комплекс PCIe. NVLink обеспечил радикальное расширение полосы пропускания обмена данными, что является обязательным условием для

эффективного распределенного обучения моделей (model parallelism) в кластерах.

- **Вытесняющая многозадачность (Compute/Instruction-Level Preemption)**

Аппаратный механизм тонкогранулярного планирования, позволяющий прерывать выполнение вычислительных задач (compute kernels) на уровне отдельных инструкций процессора. При прерывании GPU аппаратно сохраняет полный контекст выполнения задачи в память, уступает ресурсы более приоритетному процессу, а затем восстанавливает контекст и продолжает работу. Это гарантирует предсказуемую задержку отклика системы и устраняет проблему монополизации мультипроцессоров длительными ядрами.

2.1.6 Архитектура Maxwell (2014)

Архитектура Maxwell стала знаковым этапом в развитии графических процессоров NVIDIA, сместив акцент с простого наращивания количества вычислительных блоков на радикальное повышение энергетической эффективности и оптимизацию использования ресурсов каждого транзистора. Основные инновации сосредоточены в области внутренней логики планирования и глубокой модернизации подсистемы памяти.

Реорганизация потоковых мультипроцессоров (SMM)

В архитектуре Maxwell была представлена новая структура мультипроцессора — SMM (Maxwell Streaming Multiprocessor), разработанная для достижения максимальной производительности на ватт:

- **Секционирование и логика планирования:** Каждый SMM разделен на четыре отдельных блока обработки, каждый из которых обладает собственным буфером инструкций, планировщиком (warp scheduler) и диспетчером. В отличие от предыдущих итераций, такая жесткая привязка ресурсов к конкретным группам CUDA-ядер позволила упростить логику управления и существенно снизить энергопотребление накладных расходов планирования.
- **Эффективность CUDA-ядер:** Несмотря на меньшее количество ядер на один SM (128 против 192 в Kepler), каждый SMM обеспечивает значительно более высокую полезную нагрузку за счет оптимизации путей передачи инструкций и улучшенного заполнения конвейеров, что позволяет одному ядру Maxwell выполнять до 1.35 раза больше работы при той же частоте.
- **Разделяемая память и кэш L1:** В SMM разделяемая память (Shared Memory) была физически отделена от кэша L1, что увеличило доступный объем ресурсов для высокопараллельных задач и устранило конфликты при распределении памяти, характерные для комбинированных блоков.

Оптимизация подсистемы памяти и сжатия данных

Для эффективной работы в условиях ограниченной ширины шины памяти, архитектура Maxwell внедряет передовые методы сокращения избыточности данных:

- **Третье поколение дельта-сжатия цвета (Delta Color Compression):** Графический процессор получил аппаратную поддержку более эффективных алгоритмов сжатия без потерь. Это позволяет записывать в видеопамять только разницу (дельту) между значениями соседних пикселей, что эффективно увеличивает пропускную способность памяти до 25-30% за счет снижения объема передаваемого трафика.
- **Масштабированный кэш второго уровня (L2 Cache):** В архитектуре реализовано значительное увеличение объема кэша L2 (до 2 МБ в чипах среднего сегмента, что в 8 раз больше предшественников). Это позволяет удерживать значительно больше рабочих данных на кристалле, сокращая количество энергозатратных обращений к внешней памяти VRAM.

Специализированные аппаратные блоки и алгоритмы освещения

Архитектура Maxwell привнесла новые возможности для вычислений в реальном времени и обработки медиаконтента:

- **Voxel Global Illumination (VXGI):** Аппаратная поддержка алгоритмов воксельного глобального освещения. Процессор способен эффективно переводить геометрию сцены в воксельную структуру для быстрого просчета отраженного света и мягких теней.
- **Обновленный NVENC:** Интеграция выделенного кодировщика с поддержкой формата H.265 (HEVC). Это позволило аппаратно обрабатывать видео сверхвысокого разрешения (4K) с минимальными задержками и без нагрузки на основные вычислительные CUDA-ядра.
- **Multi-Pixel Programming Sampling:** Технология, оптимизирующая работу блоков растеризации для поддержки новых режимов сглаживания и повышения качества выборки пикселей при рендеринге сложных сцен.

2.1.7 Архитектура Tesla (2006)

Архитектура Tesla представляет собой первую в индустрии полностью унифицированную вычислительную архитектуру графических процессоров. Главным концептуальным сдвигом стал отказ от отдельных аппаратных блоков для обработки вершинных и пиксельных шейдеров в пользу массива универсальных скалярных процессоров, способных выполнять любой тип вычислительной нагрузки.

Унифицированная архитектура и скалярная обработка

В основе Tesla лежит отказ от традиционной векторной обработки данных в пользу мелкозернистой скалярной архитектуры:

- **Унифицированные шейдерные ядра:** Вместо выделенных аппаратных ресурсов для разных стадий графического конвейера, Tesla использует единый массив потоковых процессоров (Streaming Processors, SP). Это позволяет динамически перераспределять вычислительную мощность в зависимости от текущей нагрузки, устраняя простои оборудования.
- **Скалярное исполнение инструкций:** В отличие от предыдущих архитектур, оперировавших векторами (например, RGBA), процессоры Tesla работают со скалярными величинами. Это упрощает компиляцию кода и повышает эффективность использования ресурсов, так как системе не требуется заполнять все компоненты вектора для выполнения операции.

Организация потоковых мультипроцессоров (SM)

Вычислительные ресурсы Tesla сгруппированы в иерархическую структуру, оптимизированную для параллельного выполнения тысяч потоков:

- **Потоковый мультипроцессор (SM):** Каждый SM в архитектуре Tesla содержит 8 скалярных процессоров (SP), два специальных функциональных блока (SFU) для вычисления трансцендентных функций и блок управления потоками.
- **Разделяемая память (Shared Memory):** Важнейшим нововведением стало появление локальной памяти объемом 16 КБ внутри каждого SM. Эта память имеет крайне низкие задержки и позволяет потокам внутри одного блока обмениваться данными напрямую, что фактически превратило GPU в полноценный параллельный процессор.
- **Механизм выполнения Warps:** Архитектура ввела понятие «варпа» (warp) — группы из 32 потоков, которые выполняют одну и ту же инструкцию одновременно (SIMT — Single Instruction, Multiple Threads). Это позволило скрыть задержки обращения к памяти за счет мгновенного переключения между активными группами потоков на аппаратном уровне.

Подсистема памяти и модель ввода-вывода

Для поддержки общих вычислений Tesla внедрила новые механизмы взаимодействия с системными ресурсами:

- **Глобальное адресное пространство:** Tesla обеспечила возможность прямой адресации памяти, что позволило реализовать поддержку языков программирования высокого уровня, таких как C, через программно-аппаратный стек CUDA.

- **Текстурный и константный кэши:** Несмотря на универсальность, архитектура сохранила специализированные кэши для чтения данных, оптимизированные под конкретные паттерны доступа (например, двумерную локальность в текстурах), что существенно ускоряет работу графических и научных приложений.

2.2 Вычислительные возможности Nvidia CUDA

Видеокарты NVIDIA CUDA, относящиеся к разным микроархитектурам, отличаются количеством ядер различного назначения. Далее представлены основные характеристики вычислительных возможностей различных поколений Nvidia CUDA.

Микроархитектура Fermi

- В каждом мультипроцессоре (SM) содержится:
 - 32 ядра CUDA, предназначенные для операций с целыми числами и числами с плавающей точкой;
 - 16 блоков загрузки и сохранения данных (LD/ST);
 - 4 блока специального назначения (SFU), используемые для вычисления сложных арифметических функций.
- Планирование потоков осуществляется двумя планировщиками варпов (Warp Scheduler), поддерживающими одновременное выполнение независимых инструкций (dual-issue).
- Организация памяти: регистровый файл объёмом 128 КБ; кэш первого уровня (L1) — 16 КБ или 48 КБ; разделяемая память — 48 КБ или 16 КБ (конфигурация может варьироваться).
- Ключевая особенность: базовая структура SM и поддержка dual-issue для повышения производительности.

Микроархитектура Kepler

- **Мультипроцессор SMX:** содержит 192 ядра CUDA (организованные в 12 групп по 16 ядер), 64 вычислительных блока для операций с двойной точностью (FP64), 32 блока загрузки и сохранения данных (LD/ST), а также 32 блока специального назначения (SFU).
- **Планировщики:** оснащён 4 планировщиками варпов (Warp Scheduler), каждый из которых имеет по два блока выдачи команд.

- **Память:** регистровый файл объёмом 256 КБ; кэш первого уровня (L1) может быть сконфигурирован как 16, 48 или 32 КБ; разделяемая память также доступна в вариантах 48, 16 или 32 КБ; кроме того, имеется 48 КБ кэша для констант (кэш только для чтения).

Микроархитектура Maxwell

- **Мультипроцессор SMM:** состоит из четырёх блоков, каждый из которых содержит 32 ядра CUDA, 8 блоков загрузки/сохранения (LD/ST) и 8 блоков специального назначения (SFU).
- **Память:** на каждый блок приходится 64 КБ регистрового файла; отдельно выделено 90 КБ разделяемой памяти (Shared Memory).
- **Особенности:** модульная организация, двукратное повышение производительности на ватт и в 1.4 раза — на единицу площади кристалла по сравнению с предыдущими поколениями.

Микроархитектура Pascal

- **Мультипроцессор (SM):** включает два вычислительных блока с удвоенным регистровым файлом и увеличенным количеством блоков для операций с двойной точностью (FP64).
- **Память:** 64 КБ разделяемой памяти (Shared Memory), а также объединённый кэш текстур и первого уровня (Texture/L1 cache).
- **Особенности:** переход на техпроцесс 16 нм FinFET, что даёт улучшение производительности в расчёте на ватт.

Микроархитектура Turing

- **Мультипроцессор (SM):** состоит из четырёх блоков; каждый блок включает:
 - 32 ядра для вычислений с одинарной точностью (FP32);
 - 32 ядра для целочисленных операций (INT);
 - 8 ядер для двойной точности (FP64);
 - 2 тензорных ядра (Tensor Core);
 - 8 блоков загрузки/сохранения (LD/ST);
 - 1 блок специальных функций (SFU).
- **Тензорные ядра:** предназначены для обработки матриц размером 4×4 (входные данные в формате FP16/FP32, выходные — FP32/FP64), обеспечивая ускорение до 9.3 раза в задачах глубокого обучения.

- **Память:** 128 КБ общей памяти (L1/Shared Memory), причём до 96 КБ может быть выделено под разделяемую память.
- **Особенности:** независимое планирование потоков (каждый поток имеет собственный счётчик команд), техпроцесс 12 нм FinFET, наличие тензорных ядер для задач искусственного интеллекта.

2.3 Потокковая модель

Вычислительная платформа CUDA базируется на идее мультипроцессора и модели исполнения SIMT (Single Instruction Multiple Threads) — одна команда управляет множеством потоков.

При работе многопоточной программы на графическом процессоре CUDA все потоки разбиваются на блоки, а внутри блоков — на варпы (группы по 32 потока), причём все потоки одного варпа выполняют одну и ту же команду. Группа блоков выполняется на одном потоковом мультипроцессоре, а распределением нагрузки между ними занимается планировщик.

Программа, которая выполняется одновременно на нескольких блоках, называется ядром (kernel).

Отличительной чертой архитектуры CUDA является блочно-сеточная организация, которая нехарактерна для обычных многопоточных приложений. Эта организация показана на рисунке 1. При этом драйвер CUDA самостоятельно управляет распределением ресурсов устройства между потоками.

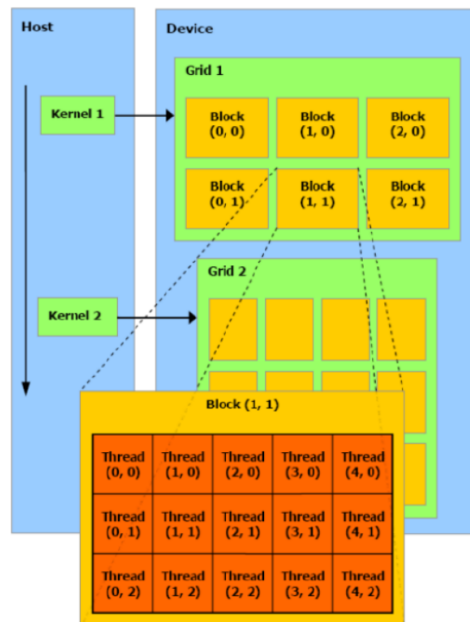


Рис. 1: Организация потоков

2.4 Устройство памяти

Графический процессор оснащён собственной глобальной памятью, которая не совпадает с оперативной памятью центрального процессора.

При выполнении кода на видеокарте обращение возможно лишь к её собственной памяти. Следовательно, для проведения вычислений и получения итогового результата требуется скопировать данные из оперативной памяти CPU в память GPU, а затем — обратно. Кроме того, каждый мультипроцессор помимо глобальной памяти располагает собственными кэшами, а также текстурной, константной, разделяемой памятью и так далее. Структура организации памяти в программах CUDA показана на рисунке 2.

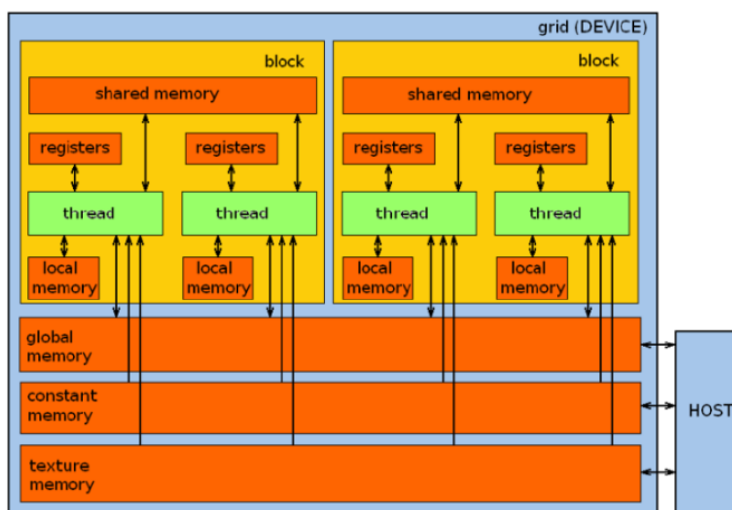


Рис. 2: Схема организации памяти CUDA-программы

2.5 Модели памяти

В архитектуре CUDA различают шесть типов памяти:

1. Регистры

Компилятор по возможности старается размещать все локальные переменные функций именно в регистровой памяти. На каждый мультипроцессор приходится 8192 32-разрядных регистров, которые распределяются между нитями, выполняющимися на этом мультипроцессоре. Доступ к регистрам является самым быстрым.

2. Локальная память

Это небольшой объем памяти, доступный только одному потоковому процессору. В неё попадают локальные данные, имеющие слишком большой размер. Физически локальная память устроена так же, как глобальная, и работает с той же скоростью.

3. Глобальная память

Глобальная память обладает большим объёмом (от 256 Мбайт до 4 Гбайт). Она доступна для произвольного чтения и записи всем мультипроцессорам, а также центральному процессору (хосту). Однако эта память очень медленная и не кэшируется, поэтому количество обращений к ней следует минимизировать. Глобальная память физически расположена на хосте, тогда как разделяемая память — непосредственно на видеокарте. Из-за этого накладные расходы на передачу данных по шине для глобальной памяти выше, чем для разделяемой, хотя сами типы памяти по своей природе идентичны.

4. Разделяемая память

Разделяемая память не кэшируется, но работает быстро. Её рекомендуется использовать в качестве управляемого кэша. На один мультипроцессор выделяется всего 16 КБ разделяемой памяти. Она служит для взаимодействия между потоками, управляется непосредственно разработчиком и обладает малой задержкой. Отличительная особенность: разделяемая память адресуется одинаково для всех задач внутри блока, то есть её можно применять для обмена данными только между потоками одного блока.

5. Константная память

Константная память хранит неизменяемые данные, доступные на уровне всей сетки (grid). Её объём составляет 64 КБ, и физически она не отделена от глобальной памяти. Однако благодаря использованию кэшей и механизма широковещательной рассылки (broadcast) этот тип памяти может повысить производительность за счёт снижения трафика между процессором и памятью. Из-за небольшого объёма в ней имеет смысл размещать лишь небольшое количество часто используемых данных.

6. Текстурная память

Текстурная память похожа на константную: через текстурный блок возможны только операции чтения. Физически она также не отделена от глобальной. Как и константная, позволяет ускорить работу благодаря системе кэшей. Её отличительная черта — оптимизация текстурного кэша для двумерной пространственной локальности.

При вычислениях на модели в виде матрицы, когда соседние элементы взаимодействуют друг с другом, часто возникает необходимость обращаться к элементам окрестности некоторого элемента матрицы. С точки зрения адресации, элементы окрестности не расположены в памяти рядом друг с другом. Текстурная память позволяет кэшировать данные,

основываясь на их пространственной, а не адресной локальности, то есть на том, что данные расположены близко в двумерном пространстве.

2.6 Модель вычислений на GPU

Программа, предназначенная для выполнения на GPU с поддержкой NVIDIA CUDA, называется ядром (kernel). Ядро запускается одновременно на сетке, состоящей из блоков, каждый из которых включает несколько нитей (потоков). Количество блоков и число нитей в блоке задаются в момент вызова ядра. Все нити выполняют один и тот же код. Чтобы обеспечить разное поведение отдельных нитей, используется информация о номере блока и нити, которая доступна каждой нити.

С точки зрения графического процессора, понятия «поток» не существует — все команды, поступающие на GPU, обрабатываются в единой очереди для всех потоков.

Планировщик способен сглаживать порядок выполнения команд, одновременно запуская копирование данных с хоста, выполнение ядра и обратное копирование на хост. Если ядро загружает менее 50% вычислительных ресурсов GPU, планировщик может запустить следующее ядро из другого потока, если оно готово к исполнению.

Для оптимизации времени выполнения задач в разных потоках важно учитывать, что команды на GPU выполняются в том порядке, в котором они были вызваны на хосте. Это означает, что не всегда эффективно полностью заполнять один поток задачами, затем второй и так далее. Более рациональный подход — равномерно распределять задачи по всем потокам.

Например, можно сначала отправить все команды в первый поток, а затем во второй. В таком случае команды второго потока будут ждать завершения команд первого. Альтернативный вариант — чередовать команды: отправить первую команду в первый поток, первую — во второй, затем вторую — в первый, вторую — во второй и так далее. Такое равномерное распределение позволяет повысить производительность, поскольку планировщик может одновременно запускать операции копирования и выполнения ядер.

2.7 Компиляция программы

Программы для видеокарт NVIDIA CUDA создаются на основе других языков программирования, в частности, используется расширение языка C, известное как CUDA C.

Для компиляции таких программ применяется компилятор nvcc, который входит в комплект инструментов разработчика. Данный набор инструментов, а также библиотеку CUDA можно загрузить с официального сайта NVIDIA.

2.8 Планировщик задач

Поток (stream) в CUDA представляет собой логическую цепочку зависимых асинхронных операций, которые выполняются независимо от операций в других потоках. Благодаря потокам можно отправлять команды CUDA на графический процессор в том порядке, который определён внутри контекста каждого отдельного потока.

С точки зрения GPU, потоки не существуют как самостоятельные объекты — все команды, поступающие на устройство, выстраиваются в общую для всех потоков очередь. Знание этой особенности может помочь при оптимизации производительности.

Планировщик способен сглаживать порядок исполнения: он может одновременно запускать копирование данных с хоста на устройство, копирование с устройства на хост и выполнение ядра. При вызове ядра можно указать, в какой поток его поместить. По умолчанию все ядра направляются в поток 0, который является синхронным по отношению к хосту. Если ядро загружает менее 50% вычислительных ресурсов GPU, планировщик может запустить следующее ядро из другого потока при условии его готовности к выполнению.

Планировщик задач в CUDA — это механизм, управляющий исполнением потоков на GPU. Он отвечает за распределение ресурсов (вычислительных ядер, памяти и т.д.) между потоками и блоками. На рисунке 3 показан планировщик задач для архитектуры Kepler.

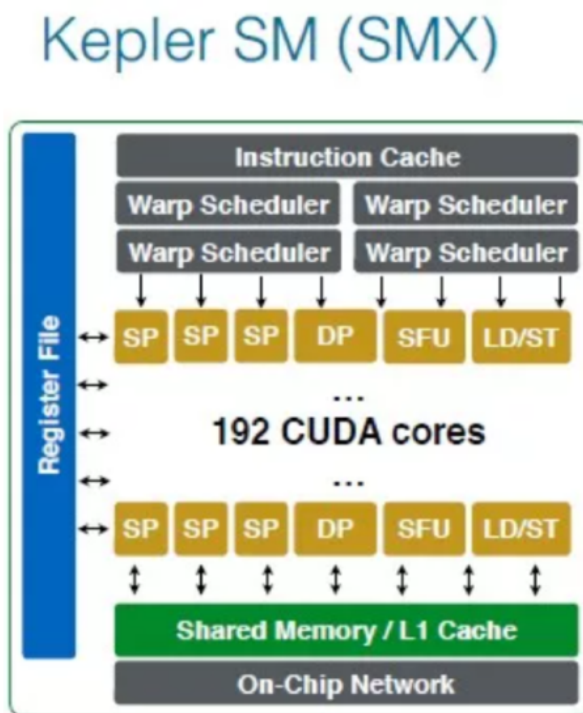


Рис. 3: Планировщик задач в архитектуре Kepler

3 Суперкомпьютерный центр «Политехнический»

3.1 Состав

В настоящий момент пользователям суперкомпьютерного центра «Политехнический» предоставлены следующие вычислительные мощности:

- 612 узлов кластера «Политехник – РСК Торнадо»;
- 56 узлов кластера «Политехник – РСК Торнадо», оснащённых ускорителями NVIDIA K-40;
- 288 узлов вычислительной системы с высокой степенью многопоточности «Политехник – РСК ПетаСтрим».

3.2 Характеристики

- Один узел кластера «Политехник – РСК Торнадо» оснащён:
 - двумя процессорами Intel Xeon E5-2697 v3 с тактовой частотой 2.60 ГГц;
 - 64 ГБ оперативной памяти.
- Один узел кластера «Политехник – РСК Торнадо» с графическими ускорителями NVIDIA K-40 включает:
 - два процессора Intel Xeon E5-2697 v3 @ 2.60 ГГц;
 - 64 ГБ RAM;
 - два ускорителя Nvidia Tesla K40x с 12 ГБ видеопамяти GDDR.
- Один узел вычислителя «Политехник – РСК ПетаСтрим» содержит:
 - один процессор Intel Xeon Phi 5120D с частотой 1.10 ГГц;
 - 8 ГБ оперативной памяти.

В качестве коммуникационной сети на всех доступных узлах используется InfiniBand со скоростью 56 Гбит/с (FDR). Кроме того, на всех узлах доступна параллельная файловая система Lustre объёмом около 1 петабайта.

По умолчанию пользователю предоставляется доступ к узлам **tornado**. Доступ к узлам других типов выдаётся по отдельному запросу.

3.3 Технология подключения

Для подключения зарегистрированного пользователя к СКЦ необходимо использовать терминальный клиент, поддерживающий протокол SSH. С помощью него получается доступ к удалённому терминалу для работы с ресурсами СКЦ.

В рамках работы была использована следующая технология подключения:

1. От администрации СКЦ были получены публичный и приватный ключи в виде файлов, а также логин пользователя `tm4u10` для доступа к системе.
2. Установка программного обеспечения:

- Для удобства переноса файлов между персональным компьютером и узлом СКЦ был установлен клиент WinSCP.
- Для организации подключения к терминалу СКЦ необходимо установить инструмент OpenSSH.

На ноутбуке с операционной системой Windows 10 OpenSSH предустановлен, однако нужно вручную запустить службу `ssh-agent`. Для этого нужно запустить `cmd` от имени администратора и выполнить команды.

```
1 Set-Service ssh-agent-StartupType Automatic
2 Start-Service ssh-agent
```

Далее нужно добавить ключ

```
1 ssh-add path/to/id_rsa
2 ssh-add -l
```

3. Подключение к узлу СКЦ через SSH: выполнить команду

```
1 ssh -i .ssh/private-key login@login1.hpc.spbstu.ru
```

где:

- `login` – логин для подключения к СКЦ;
- `login@login1.hpc.spbstu.ru` – адрес узла СКЦ;
- `.ssh/private-key` – путь до приватного файла ключа.

4. Настройка WinSCP для передачи файлов: настроить следующие параметры

- Для входа:
 - Имя хоста: login1.hpc.spbstu.ru
 - Порт: 22
 - Имя пользователя: tm4u10
- Для аутентификации в разделе «Ещё... -> SSH -> Аутентификация» был выбран файл приватного ключа, автоматически преобразованный в формат .ppk. На рисунках 4, 5 показана последовательность действий.

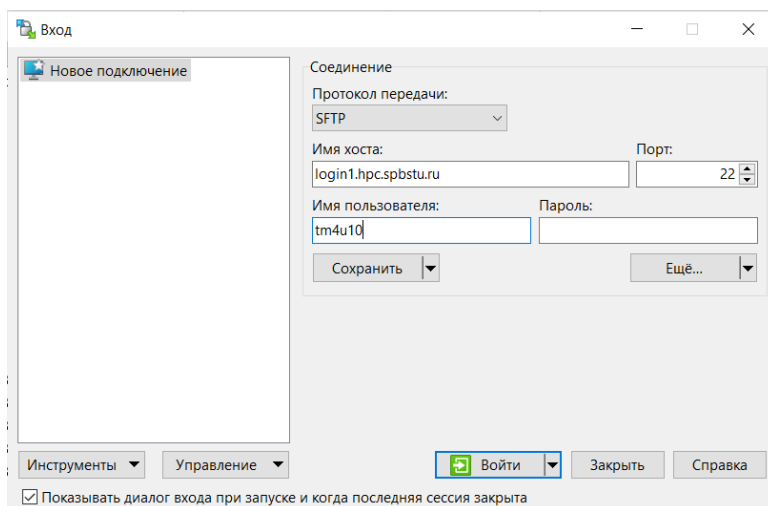


Рис. 4: Настройка WinSCP

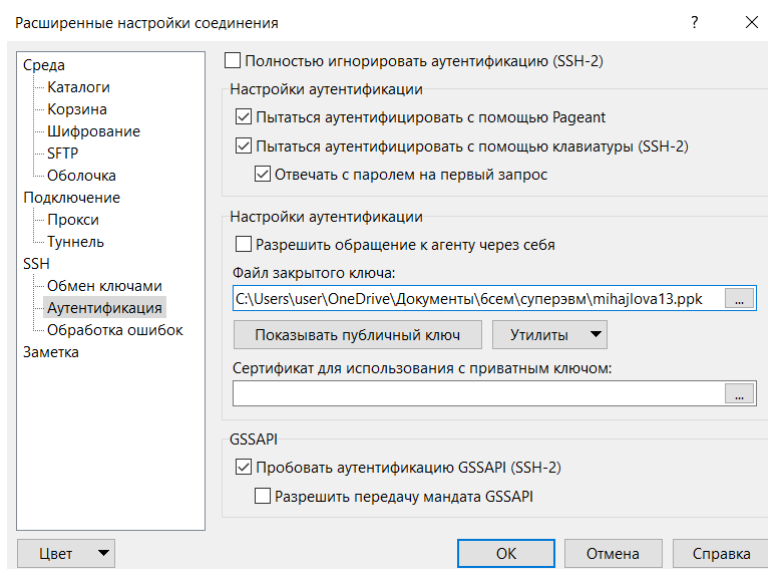


Рис. 5: Настройка WinSCP

В результате было осуществлено подключение к СКЦ по протоколу SSH. На рисунке 6 показана стартовая страница при подключении к СКЦ.

```

Last login: Tue Feb 24 13:12:36 2026

/##### /##### /##### /##### /##### /$ /##### /$ /$
/$_ $$ /$_ $$ /$_ $$ /$_ $$ |$_ $| $$_ $| $$_ $| $$_ $|
| $$ \_ / | $$ \_ / | $$ \_ / | $$ \_ / | $$$$$$ | $$ \_ $| $$ \_ $|
| $$$$$$ | $$ | $$ | $$$$$$ | $$$$$$/ | $$_ $$ | $$$$$$/ | $$ | $$
\_ $$ | $$ | $$ | $$_ $| $$_ $| $$_ $| $$_ $|
/$$ \ $$ | $$ $$ | $$ $$ /$$ \ $$ | $$ | $$ | $$ | $$ | $$
| $$$$$$/ | $$$$$$/ | $$$$$$/ | $$$$$$/ | $$$$$$/ | $$$$$$/
\_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \_

Slurm partitions:
* tornado : nodes: 612
    cpu: 2 x Intel Xeon CPU E5-2697 v3 @ 2.60GHz
    cores/hwthreads: 28 / 56
    mem: 64G
    net: 56Gbps FDR Infiniband
* tornado-k40 : nodes: 56
    cpu: 2 x Intel Xeon CPU E5-2697 v3 @ 2.60GHz
    cores/hwthreads: 28 / 56
    co-processor: 2 x Nvidia Tesla K40x
    co-processor mem: 12G
    mem: 64G
    net: 56Gbps FDR Infiniband

Storage: 1 PB Lustre FS

List of available software: module avail

tm4u10@login1:~
$ █

```

Рис. 6: Подключение к СКЦ

4 Постановка решаемой практической задачи

Дано:

- Массив M - матрица $n \times n$

Требуется: вычислить M^k — результат возведения матрицы в степень k .

Ограничения:

- Числа в матрице не должны превышать 1000;
- Числа в матрице не должны быть меньше -1000.

5 Алгоритм решения задачи

Для решения задачи применялся алгоритм бинарного возведения в степень:

1. Если n чётно, то верно $a^n = a^{n/2} \cdot a^{n/2} = (a^{n/2})^2$.
2. Если n нечётно, то верно $a^n = a^{n-1} \cdot a = (a^{\frac{n-1}{2}})^2 \cdot a$, то есть можно свести задачу к размеру $n - 1$ (к чётной форме).

Возведение матрицы в степень — это ее многократное умножение саму на себя:

$$A^n = \underbrace{A \cdot A \cdot \dots \cdot A}_{n \text{ раз}}$$

При этом для $n = 0$: $A^0 = E$, где E — единичная матрица.

Пусть даны две квадратные матрицы A и B размера $n \times n$:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}, \quad B = \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{pmatrix}$$

Произведение матриц $C = A \times B$ есть матрица C размера $n \times n$, элементы которой вычисляются по формуле:

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj}, \quad \text{где } i, j = 1, 2, \dots, n.$$

5.1 Метод распараллеливания алгоритма

Рассмотрим два подхода к распараллеливанию этой задачи на CUDA: с использованием глобальной памяти и с использованием разделяемой (shared) памяти.

5.1.1 Глобальная память

1. Распределение потоков

- Размер матрицы равен $n \times n$, тогда общее число элементов матрицы равно N . Запускается одномерная 1D-сетка потоков.
- Каждый поток обрабатывает несколько элементов результирующей матрицы с шагом s , где

$$s = \frac{N}{\text{gridDim.x} \cdot \text{blockDim.x}},$$

где $gridDim.x$ — число блоков, а $blockDim.x$ — число потоков внутри одного блока.

- Индекс потока вычисляется как

$$threadId = blockIdx.x \cdot blockDim.x + threadIdx.x$$

- Каждый поток обрабатывает элементы с индексами

$$idx = threadId + i \cdot T, \quad i = 0, 1, 2, \dots, \quad \text{при условии } idx < N.$$

- Каждый поток копирует элементы между матрицами, используя только глобальную память.

2. Умножение матриц

- Координаты элементов вычисляются как:

$$row = \frac{idx}{n}, \quad col = idx \bmod n$$

- Каждый поток вычисляет один элемент результирующей матрицы:

$$C[row][col] = \sum_{k=0}^{n-1} A[row][k] \cdot B[k][col]$$

5.1.2 Разделяемая память

1. Разбиение матрицы на блоки данных

- Запускается одномерная 1D-сетка потоков, которая делится на одномерные блоки потоков.
- В каждом блоке выделяется разделяемая память. Она разбивается на два буфера:
 - $rowCacheA$ — для хранения части строки матрицы A ;
 - $colCacheB$ — для хранения части столбца матрицы B .
- Размер каждого буфера составляет $blockDim.x$.

2. Копирование данных в разделяемую память

- Каждый поток загружает свою часть элементов параллельно другим потокам.
- После загрузки данных производится их синхронизация. Это гарантирует, что все элементы матрицы будут помещены в разделяемую память до начала вычислений.

- После синхронизации каждый поток вычисляет частичную сумму скалярного произведения, используя значения из разделяемой памяти. После завершения вычислений выполняется повторная синхронизация.

3. Копирование данных в глобальную память

- После всех вычислений потоки параллельно копируют данные из разделяемой памяти в глобальную. Этот процесс аналогичен предыдущему этапу.

6 Описание эксперимента

Цель эксперимента — оценка зависимости времени решения задачи от объема обрабатываемых данных и параметров запуска, связанных с GPU.

Для исследования времени решения задачи в зависимости от параметров были выбраны следующие параметры:

- Используемая память: глобальная, разделяемая и глобальная.
- Количество блоков: 1, 10, 100, 1000, 10000.
- Количество потоков в блоке: 1, 10, 100, 1000.
- Размеры матрицы: 10×10 , 100×100 , 1000×1000 .
- Количество элементов в матрице: 100, 10000, 1000000.

Для каждой комбинации параметров было проведено 100 измерений с помощью событий CUDA и взято среднее значение.

7 Анализ результатов

На рисунках 7, 8 представлены результаты эксперимента.

```
tm4u10@login1:~/prog
$ cat file-7121514.out
>> Matrix size: 10 x 10
Blocks: 1, Threads: 1 -> Time: 0.095 ms
Blocks: 1, Threads: 10 -> Time: 0.067 ms
Blocks: 1, Threads: 100 -> Time: 0.070 ms
Blocks: 1, Threads: 1000 -> Time: 0.070 ms
Blocks: 10, Threads: 1 -> Time: 0.069 ms
Blocks: 10, Threads: 10 -> Time: 0.062 ms
Blocks: 10, Threads: 100 -> Time: 0.067 ms
Blocks: 10, Threads: 1000 -> Time: 0.063 ms
Blocks: 100, Threads: 1 -> Time: 0.071 ms
Blocks: 100, Threads: 10 -> Time: 0.064 ms
Blocks: 100, Threads: 100 -> Time: 0.064 ms
Blocks: 100, Threads: 1000 -> Time: 0.061 ms
Blocks: 1000, Threads: 1 -> Time: 0.070 ms
Blocks: 1000, Threads: 10 -> Time: 0.068 ms
Blocks: 1000, Threads: 100 -> Time: 0.063 ms
Blocks: 1000, Threads: 1000 -> Time: 0.067 ms
Blocks: 10000, Threads: 1 -> Time: 0.069 ms
Blocks: 10000, Threads: 10 -> Time: 0.062 ms
Blocks: 10000, Threads: 100 -> Time: 0.063 ms
Blocks: 10000, Threads: 1000 -> Time: 0.072 ms
Matrix size: 100 x 100
Blocks: 1, Threads: 1 -> Time: 5.190 ms
Blocks: 1, Threads: 10 -> Time: 0.705 ms
Blocks: 1, Threads: 100 -> Time: 0.301 ms
Blocks: 1, Threads: 1000 -> Time: 0.346 ms
Blocks: 10, Threads: 1 -> Time: 5.187 ms
Blocks: 10, Threads: 10 -> Time: 0.702 ms
Blocks: 10, Threads: 100 -> Time: 0.298 ms
Blocks: 10, Threads: 1000 -> Time: 0.341 ms
Blocks: 100, Threads: 1 -> Time: 5.206 ms
Blocks: 100, Threads: 10 -> Time: 0.688 ms
Blocks: 100, Threads: 100 -> Time: 0.301 ms
Blocks: 100, Threads: 1000 -> Time: 0.347 ms
Blocks: 1000, Threads: 1 -> Time: 5.197 ms
Blocks: 1000, Threads: 10 -> Time: 0.695 ms
Blocks: 1000, Threads: 100 -> Time: 0.299 ms
Blocks: 1000, Threads: 1000 -> Time: 0.355 ms
Blocks: 10000, Threads: 1 -> Time: 5.185 ms
Blocks: 10000, Threads: 10 -> Time: 0.688 ms
Blocks: 10000, Threads: 100 -> Time: 0.292 ms
Blocks: 10000, Threads: 1000 -> Time: 0.344 ms
Matrix size: 1000 x 1000
Blocks: 1, Threads: 1 -> Time: 6485.322 ms
Blocks: 1, Threads: 10 -> Time: 702.370 ms
Blocks: 1, Threads: 100 -> Time: 268.095 ms
Blocks: 1, Threads: 1000 -> Time: 203.430 ms
Blocks: 10, Threads: 1 -> Time: 6485.100 ms
Blocks: 10, Threads: 10 -> Time: 702.408 ms
Blocks: 10, Threads: 100 -> Time: 268.059 ms
Blocks: 10, Threads: 1000 -> Time: 200.572 ms
```

Рис. 7: Результаты эксперимента

Blocks: 100, Threads: 1 -> Time: 6485.484 ms
Blocks: 100, Threads: 10 -> Time: 702.395 ms
Blocks: 100, Threads: 100 -> Time: 268.476 ms
Blocks: 100, Threads: 1000 -> Time: 202.892 ms
Blocks: 1000, Threads: 1 -> Time: 6485.415 ms
Blocks: 1000, Threads: 10 -> Time: 702.365 ms
Blocks: 1000, Threads: 100 -> Time: 268.092 ms
Blocks: 1000, Threads: 1000 -> Time: 204.608 ms
Blocks: 10000, Threads: 1 -> Time: 6485.268 ms
Blocks: 10000, Threads: 10 -> Time: 702.351 ms
Blocks: 10000, Threads: 100 -> Time: 268.184 ms
Blocks: 10000, Threads: 1000 -> Time: 203.998 ms

Рис. 8: Результаты эксперимента

В таблицах 1, 2, 3 представлены усреднённые результаты измерения возведения матрицы в степень 64 с использованием глобальной памяти.

Таблица 1: Усреднённые результаты измерения времени исполнения программы для матрицы 10x10 и глобальной памяти (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	0.095	0.069	0.071	0.070	0.069
10	0.067	0.062	0.064	0.068	0.062
100	0.070	0.067	0.064	0.063	0.063
1000	0.070	0.063	0.061	0.067	0.072

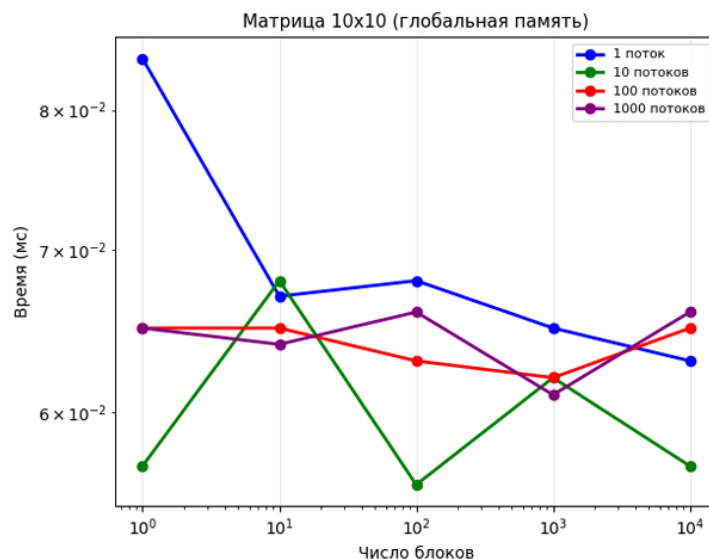


Рис. 9: График результатов времени исполнения программы для матрицы 10x10

При малом размере матрицы изменение количества блоков практически не влияет на время выполнения (0.061-0.095 мс), так как накладные расхо-

ды на организацию блоков и синхронизацию преобладают над временем вычислений. Все данные помещаются в кэш, поэтому обращения к медленной глобальной памяти минимальны.

Таблица 2: Усреднённые результаты измерения времени исполнения программы для матрицы 100x100 и глобальной памяти (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	5.190	5.187	5.206	5.197	5.185
10	0.705	0.702	0.688	0.695	0.688
100	0.301	0.298	0.301	0.299	0.292
1000	0.346	0.341	0.347	0.355	0.344

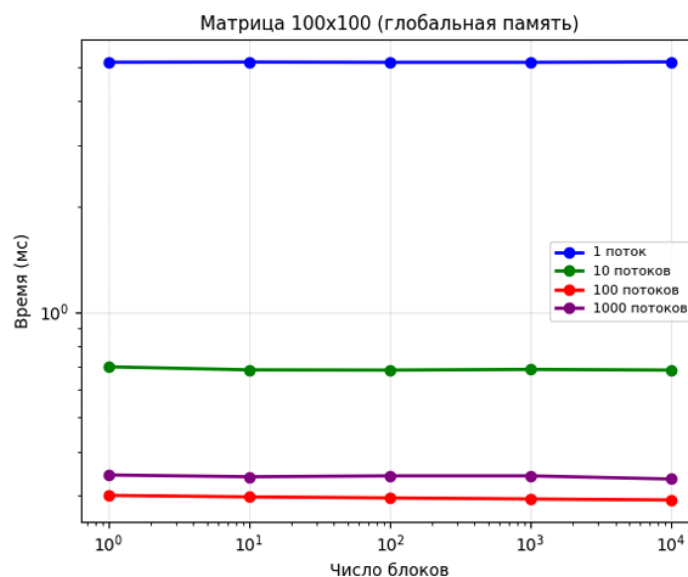


Рис. 10: График результатов времени исполнения программы для матрицы 100x100

При увеличении числа блоков с 1 до 10-100 время значительно снижается (с 5.190 до 0.292 мс при 100 потоках), так как нагрузка распределяется между мультипроцессорами. Однако дальнейшее увеличение блоков до 1000-10000 не даёт эффекта из-за ограниченной пропускной способности глобальной памяти и накладных расходов на планирование.

Таблица 3: Усреднённые результаты измерения времени исполнения программы для матрицы 1000x1000 и глобальной памяти (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	6485.322	6485.100	6485.484	6485.415	6485.268
10	702.370	702.408	702.395	702.365	702.351
100	268.095	268.059	268.476	268.092	268.184
1000	203.430	200.572	202.892	204.608	203.998

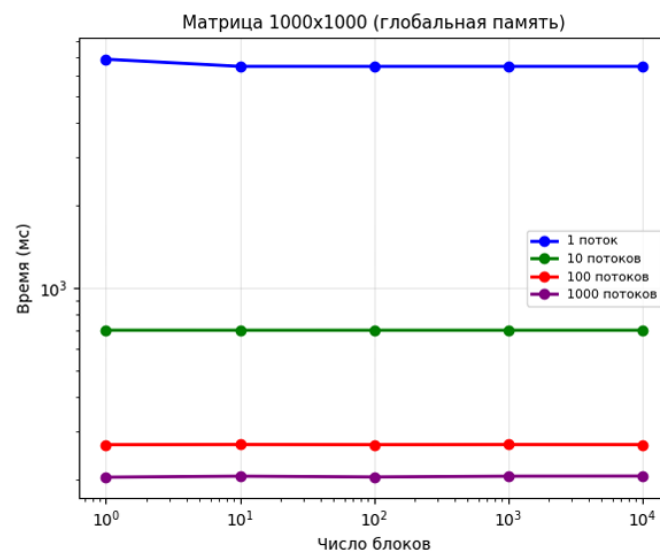


Рис. 11: График результатов времени исполнения программы для матрицы 1000x1000

При большом размере матрицы увеличение количества блоков с 1 до 1000 существенно ускоряет выполнение (с 6485 до 200 мс при 1000 потоках), поскольку параллелизм скрывает задержки доступа к глобальной памяти.

В таблицах 4, 5, 6 представлены усреднённые результаты измерения возведения матрицы в степень 64 с использованием глобальной и разделяемой памяти.

Таблица 4: Усредненные результаты измерения времени исполнения программы для матрицы 10x10 и глобальной+разделяемой памяти (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	0.195	0.094	0.092	0.091	0.286
10	0.109	0.095	0.095	0.098	0.256
100	0.097	0.098	0.155	0.099	0.286
1000	0.093	0.096	0.101	0.180	1.104

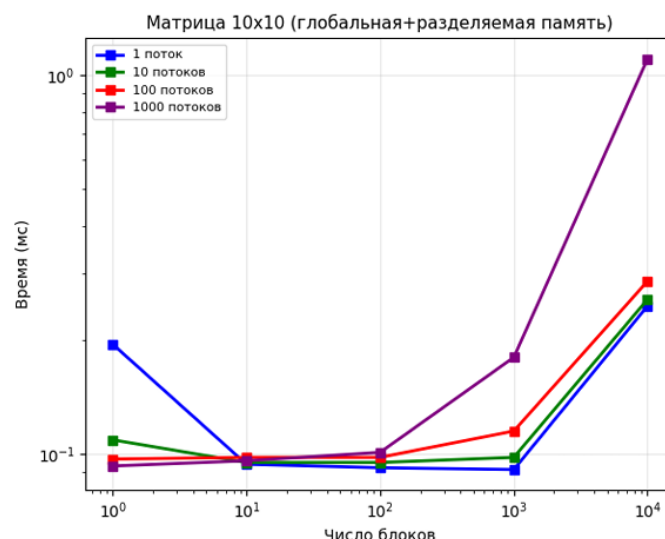


Рис. 12: График результатов времени исполнения программы для матрицы 10x10

Использование разделяемой памяти для малой матрицы неэффективно: время возрастает при большом числе блоков (до 1.104 мс при 10000 блоков), так как накладные расходы на загрузку данных в быструю разделяемую память и синхронизацию потоков внутри блока превышают выгоду от ускоренного доступа.

Таблица 5: Усредненные результаты измерения времени исполнения программы для матрицы 100x100 и глобальной+разделяемой памяти (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	0.188	0.184	0.231	0.731	4.766
10	0.185	0.192	0.192	0.661	0.735
100	0.210	0.214	0.346	0.296	0.300
1000	0.363	0.370	0.369	0.374	1.220

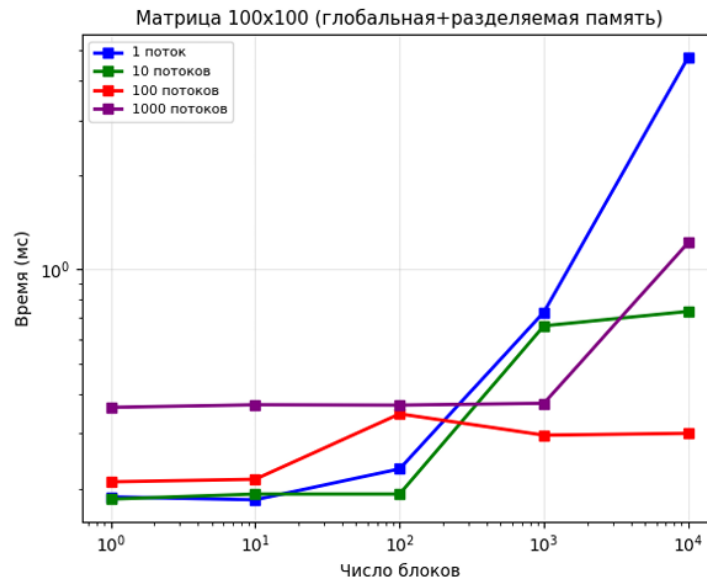


Рис. 13: График результатов времени исполнения программы для матрицы 100x100

При среднем размере матрицы оптимальное время (0.184 мс) достигается при умеренном числе блоков (10), когда данные эффективно используются из разделяемой памяти. При увеличении блоков до 1000-10000 время резко растёт (до 4.766 мс) из-за фрагментации данных в разделяемой памяти и избыточной синхронизации между потоками блока.

Таблица 6: Усредненные результаты измерения времени исполнения программы для матрицы 1000x1000 и глобальной+разделяемой памяти (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	1.338	1.292	1.323	6.752	56.191
10	1.363	1.347	1.582	7.266	70.151
100	1.597	1.614	2.703	26.304	273.540
1000	2.972	2.918	20.302	205.127	204.668

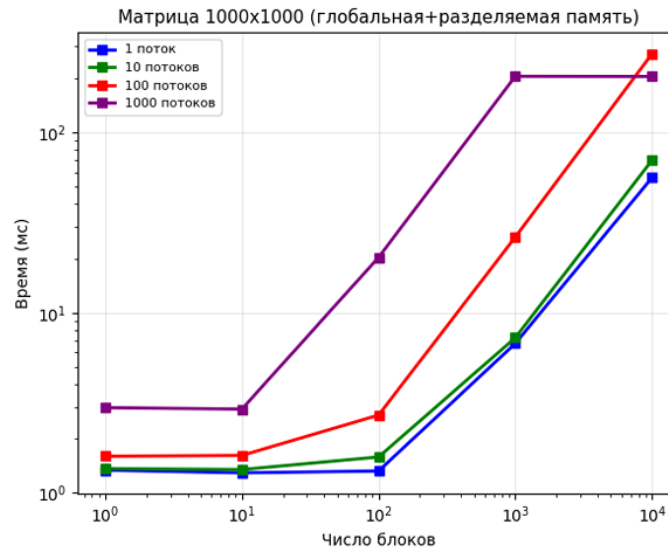


Рис. 14: График результатов времени выполнения программы для матрицы 1000x1000

Для большой матрицы разделяемая память даёт максимальный выигрыш (1.292 мс при 10 блоках), так как многократное использование элементов матрицы из быстрой разделяемой памяти (вместо глобальной) существенно снижает задержки. Однако при чрезмерном увеличении блоков (10000) время катастрофически растёт (до 273 мс) из-за нехватки разделяемой памяти на мультипроцессоре и частых обращений к глобальной памяти.

Зависимость от размера матрицы

Для всех матриц время выполнения увеличивается с увеличением размера матрицы, так как становится больше данных и больше вычислений. Для малых размеров матрицы значительную часть времени составляют накладные расходы.

- При малых размерах $n = 10$ минимальное время составляет 0.061 мс (Таблица 1).
- При средних размерах $n = 1000$ минимальное время составляет 0.184 мс (Таблица 5).
- При больших размерах $n = 1000$ минимальное время составляет 1.292 мс (Таблица 6).

Зависимость от числа блоков и потоков

Количество потоков в блоке больше влияет на производительность, чем количество блоков. Рассмотрим на примере большой матрицы 1000x1000 (таб-

лица 3):

- При 1 потоке в блоке каждый поток вынужден обрабатывать N элементов последовательно, что приводит к минимальному использованию параллелизма и большому времени выполнения, что особенно заметно на больших матрицах. Для матрицы 1000×1000 это даёт время около 6485 мс.
- При увеличении до 10 потоков время снижается в 9–10 раз (702 мс).
- При 100 и 1000 потоках дальнейшее увеличение даёт меньший прирост (200–268 мс), так как пропускная способность глобальной памяти ограничена.

Количество блоков меньше влияет на производительность. Для всех размеров матриц изменение числа блоков при фиксированном числе потоков даёт разброс времени не более 5–10%. Например, для матрицы 100×100 при числе потоков=10 время выполнения составляет от 0.688 мс до 0.705 мс.

Наименьшее время достигается при оптимальном соотношении количества потоков и блоков в них. Например, для матрицы 10000 элементов (100 на 100) минимальное время (0.292 мс) достигается при 100 блоках и 10000 потоках в каждом блоке.

Сравнение глобальной и разделяемой памяти

Результаты сравнения двух подходов к организации памяти представлены в таблице 7.

- Для малых матриц (10×10) разделяемая память не даёт преимущества, а в некоторых конфигурациях показывает худшие результаты. Это связано с тем, что время, которое тратится на накладные расходы (загрузка данных в разделяемую память, синхронизация потоков), превышает выгоду от ускоренного доступа. При малом объёме данных задержки глобальной памяти практически не проявляются.
- Для средних матриц (100×100) разделяемая память даёт небольшое улучшение в некоторых конфигурациях. Выгода от повторного использования элементов в разделяемой памяти начинает компенсировать накладные расходы, которые на нее необходимы.
- Для больших матриц (1000×1000) разделяемая память показывает значительно лучшие результаты, чем глобальная. Это связано с временем доступа: к разделяемой памяти он намного быстрее, чем к глобальной, что при многократном использовании данных даёт значимый эффект.

Результаты сравнения двух подходов к организации памяти представлены в таблице 7.

Таблица 7: Таблица максимального и минимального времени для каждой конфигурации

размер матрицы	тип памяти	мин/макс время	время (мс)	блоки	потоки
10x10	глобальная	максимальное	0.095	1	1
		минимальное	0.061	100	1000
	глобальная + разделяемая	максимальное	1.104	10000	1000
		минимальное	0.091	1000	1
100x100	глобальная	максимальное	5.206	100	1
		минимальное	0.292	100	10000
	глобальная + разделяемая	максимальное	4.766	10000	1
		минимальное	0.184	10	1
1000x1000	глобальная	максимальное	6485.484	100	1
		минимальное	200.572	1	1000
	глобальная + разделяемая	максимальное	273.668	10000	100
		минимальное	1.292	10	1

Заключение

В ходе данной курсовой работы мной было изучена технология параллельного программирования на основе архитектуры Nvidia CUDA.

В теоретической части работы был проведен обзор архитектуры NVIDIA CUDA, включающий обзор её вычислительных возможностей, модели памяти и модели вычислений на графических процессорах NVIDIA.

В ходе практической части работы:

- Реализована и распараллелена программа на языке CUDA C для возведения матрицы в k -тую степень.
- Первично программа была отлажена на персональном компьютере, поддерживающем технологию NVIDIA.
- Финальная серия экспериментов была проведена на суперкомпьютере «Политехнический» с использованием вычислительного узла «Политехник - РСК Торнадо» (графический ускоритель NVIDIA K40).
- В ходе серии экспериментов были исследованы различные конфигурации параметров распараллеливания: изменялся размер исходной матрицы матрицы, количество блоков в сетке, количество потоков в блоке и используемые типы памяти.
- В результате экспериментов были получены 120 усредненных измерений.
- Проведен анализ полученных результатов.

В ходе анализа эксперимента было выявлено, что разделяемая память дает значительное ускорение при больших объемах данных, требующих быстрого доступа к памяти. При малых объемах данных эффективнее работает глобальная память.

Измерение времени работы показало, что производительность зависит от числа потоков, количества потоков в блоке и типа используемой памяти. При использовании глобальной памяти минимальное время выполнения достигается, когда произведение числа блоков на число потоков в блоке приблизительно равно количеству элементов матрицы (например, в матрице 1000×1000 оптимальная конфигурация - 1 блок на 1000 потоков). При использовании глобальной и разделяемой памяти оптимальная производительность достигается при умеренном числе блоков (10–100) и достаточном числе потоков для эффективного использования разделяемой памяти. Для матрицы 1000×1000 минимальное время 1.292 мс получено при 10 блоках и 1 потоке, что объясняется многократным использованием данных, загруженных в разделяемую память, и снижением обращений к глобальной памяти.

Список литературы

- [1] Сандерс, Д. Технология CUDA в примерах: введение в программирование графических процессоров. — М: ДМК Пресс, 2013 г — 232 с.
- [2] Краткое руководство пользователя вычислителей «Политехник - РСК Торнадо» и «Политехник - РСК Петастрим».
- [3] Суперкомпьютерный центр - Ресурсы СКЦ. // Суперкомпьютерный центр «Политехнический». [Электронный ресурс]. – Режим доступа: URL: <https://scc.spbstu.ru/resources> (дата обращения: 10.05.2026).